Topic : Memory management, Contiguous memory allocation

Objective :

Outcome :

What is Memory Management?

Memory Management is the process of controlling and coordinating computer memory, assigning portions known as blocks to various running programs to optimize the overall performance of the system.

It is the most important function of an operating system that manages primary memory. It helps processes to move back and forward between the main memory and execution disk. It helps OS to keep track of every memory location, irrespective of whether it is allocated to some process or it remains free.

Why Use Memory Management?

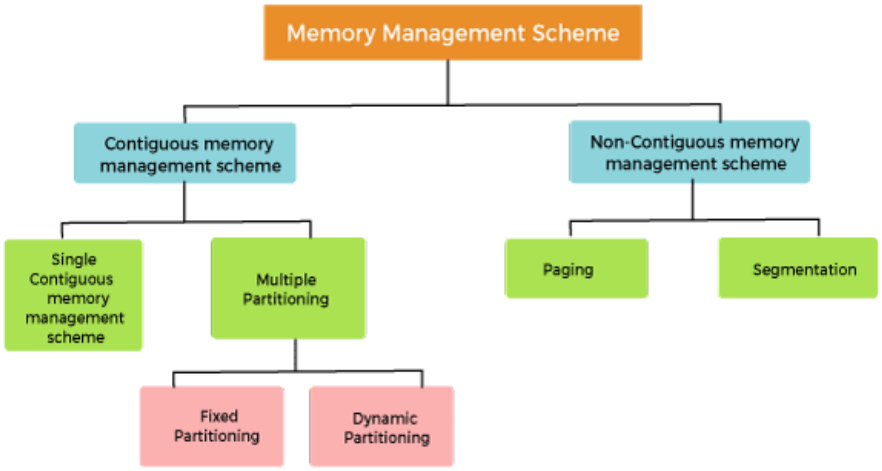
Here, are reasons for using memory management:

- It allows you to check how much memory needs to be allocated to processes that decide which processor should get memory at what time.
- Tracks whenever inventory gets freed or unallocated. According to it will update the status.
- It allocates the space to application routines.
- It also make sure that these applications do not interfere with each other.
- Helps protect different processes from each other
- It places the programs in memory so that memory is utilized to its full extent.

Memory management Techniques:

The Memory management Techniques can be classified into following main categories:

- Contiguous memory management schemes
- Non-Contiguous memory management schemes



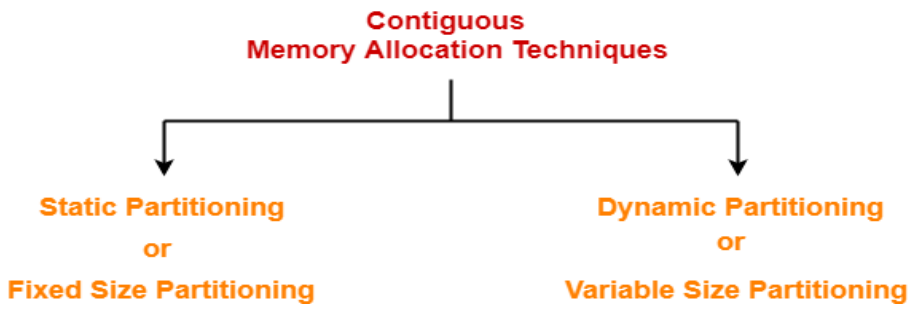
Classification of memory management schemes

CONTIGUOUS MEMORY ALLOCATION-

- Contiguous memory allocation is a memory allocation technique.
- It allows to store the process only in a contiguous fashion.
- Thus, entire process has to be stored as a single entity at one place inside the memory.

Techniques-

There are two popular techniques used for contiguous memory allocation-

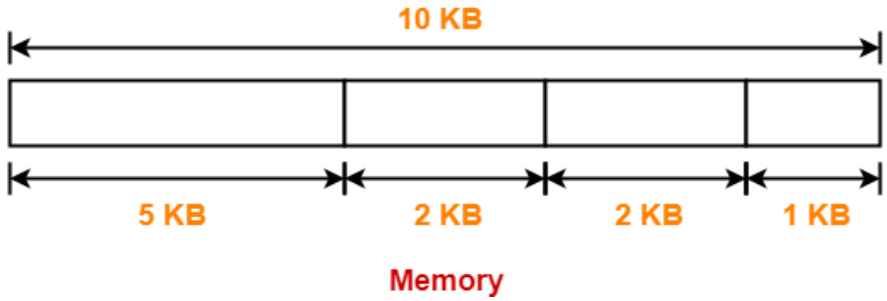


Static Partitioning-

- Static partitioning is a fixed size partitioning scheme.
- In this technique, main memory is pre-divided into fixed size partitions.
- The size of each partition is fixed and cannot be changed.
- Each partition is allowed to store only one process.

Example-

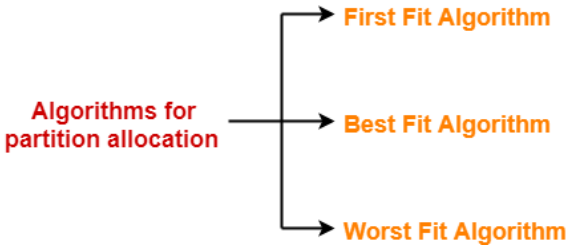
Under fixed size partitioning scheme, a memory of size 10 KB may be divided into fixed size partitions as-



- These partitions are allocated to the processes as they arrive.
- The partition allocated to the arrived process depends on the algorithm followed.

Algorithms for Partition Allocation-

Popular algorithms used for allocating the partitions to the arriving processes are-



1. First Fit Algorithm
2. Best Fit Algorithm
3. Worst Fit Algorithm

1. First Fit Algorithm-

- This algorithm starts scanning the partitions serially from the starting.
- When an empty partition that is big enough to store the process is found, it is allocated to the process.
- Obviously, the partition size has to be greater than or at least equal to the process size.

2. Best Fit Algorithm-

- This algorithm first scans all the empty partitions.
- It then allocates the smallest size partition to the process.

3. Worst Fit Algorithm-

- This algorithm first scans all the empty partitions.
- It then allocates the largest size partition to the process.

Point-01:

For static partitioning,

- Best Fit Algorithm works best.
- This is because space left after the allocation inside the partition is of very small size.
- Thus, internal fragmentation is least.

Point-02:

For static partitioning,

- Worst Fit Algorithm works worst.
- This is because space left after the allocation inside the partition is of very large size.
- Thus, internal fragmentation is maximum.

Advantages-

The advantages of static partitioning are-

- It is simple and easy to implement.
- It supports multiprogramming since multiple processes can be stored inside the main memory.
- Only one memory access is required which reduces the access time.

Disadvantages-

- It suffers from both internal fragmentation and external fragmentation.
- It utilizes memory inefficiently.
- The degree of multiprogramming is limited equal to number of partitions.
- There is a limitation on the size of process since processes with size greater than the size of largest partition can't be stored and executed.

DYNAMIC PARTITIONING

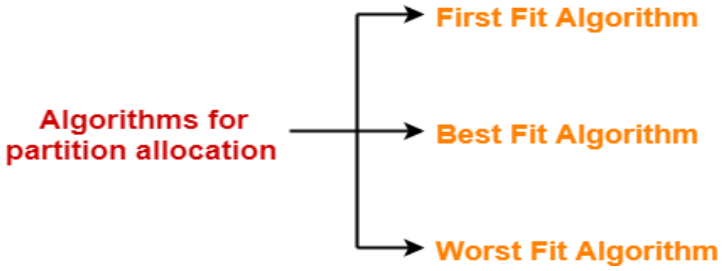
- Dynamic partitioning is a variable size partitioning scheme.
- It performs the allocation dynamically.
- When a process arrives, a partition of size equal to the size of process is created.
- Then, that partition is allocated to the process.

Partition Allocation Algorithms-

- The processes arrive and leave the main memory.
- As a result, holes of different size are created in the main memory.
- These holes are allocated to the processes that arrive in future.

Partition allocation algorithms are used to decide which hole should be allocated to the arrived process.

Popular partition allocation algorithms are-



Point-01:

For dynamic partitioning,

- Worst Fit Algorithm works best.
- This is because space left after allocation inside the partition is of large size.
- There is a high probability that this space might suit the requirement of arriving processes.

Point-02:

For dynamic partitioning,

- Best Fit Algorithm works worst.
- This is because space left after allocation inside the partition is of very small size.
- There is a low probability that this space might suit the requirement of arriving processes.

Advantages-

The advantages of dynamic partitioning are-

- It does not suffer from internal fragmentation.
- Degree of multiprogramming is dynamic.
- There is no limitation on the size of processes.

Disadvantages-

- It suffers from external fragmentation.
- Allocation and deallocation of memory is complex.

Summary -

Topic : Fragmentation

Objective :

Outcome :

What is Fragmentation?

Fragmentation is an unwanted problem in the operating system in which the processes are loaded and unloaded from memory, and free memory space is fragmented. Processes can't be assigned to memory blocks due to their small size, and the memory blocks stay unused. It is also necessary to understand that as programs are loaded and deleted from memory, they generate free space or a hole in the memory. These small blocks cannot be allotted to new arriving processes, resulting in inefficient memory use.

The conditions of fragmentation depend on the memory allocation system. As the process is loaded and unloaded from memory, these areas are fragmented into small pieces of memory that cannot be allocated to incoming processes. It is called **fragmentation**.

Causes of Fragmentation

User processes are loaded and unloaded from the main memory, and processes are kept in memory blocks in the main memory. Many spaces remain after process loading and swapping that another process cannot load due to their size. Main memory is available, but its space is insufficient to load another process because of the dynamical allocation of main memory processes.

Types of Fragmentation-

There are mainly two types of fragmentation in the operating system. These are as follows:

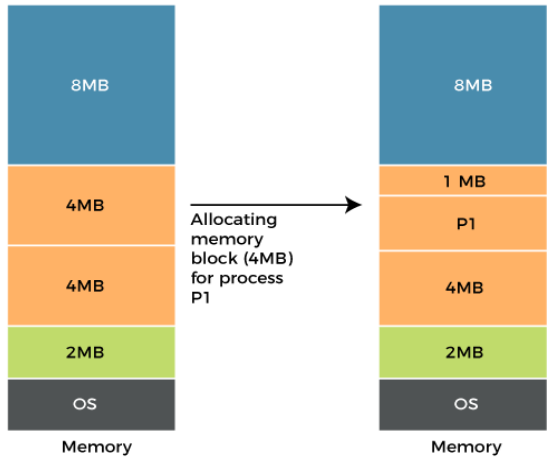
1. **Internal Fragmentation**
2. **External Fragmentation**

1.) Internal Fragmentation

When a process is allocated to a memory block, and if the process is smaller than the amount of memory requested, a free space is created in the given memory block. Due to this, the free space of the memory block is unused, which causes **internal** fragmentation.

For Example:

Assume that memory allocation in RAM is done using fixed partitioning (i.e., memory blocks of fixed sizes). **2MB, 4MB, 4MB**, and **8MB** are the available sizes. The Operating System uses a part of this RAM.



Let's suppose a process **P1** with a size of **3MB** arrives and is given a memory block of **4MB**. As a result, the **1MB** of free space in this block is unused and cannot be used to allocate memory to another process. It is known as **internal fragmentation**.

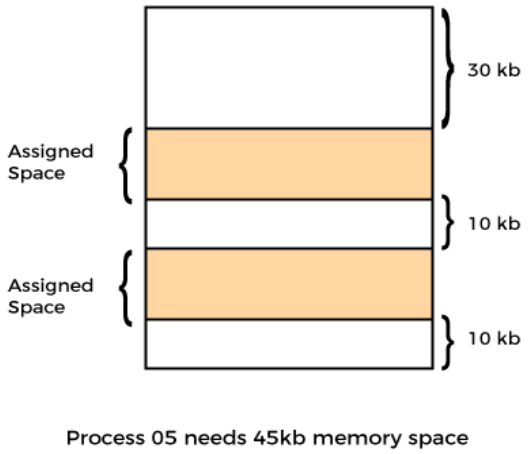
How to avoid internal fragmentation?

The problem of internal fragmentation may arise due to the fixed sizes of the memory blocks. It may be solved by assigning space to the process via dynamic partitioning. Dynamic partitioning allocates only the amount of space requested by the process. As a result, there is no internal fragmentation.

2.) External Fragmentation

External fragmentation happens when a dynamic memory allocation method allocates some memory but leaves a small amount of memory unusable. The quantity of available memory is substantially reduced if there is too much external fragmentation. There is enough memory space to complete a request, but it is not contiguous. It's known as **external** fragmentation.

For Example:



Let's take the example of external fragmentation. In the above diagram, you can see that there is sufficient space **(50 KB)** to run a process **(05) (need 45KB)**, but the memory is not contiguous. You can use compaction, paging, and segmentation to use the free space to execute a process.

How to remove external fragmentation?

This problem occurs when you allocate RAM to processes continuously. It is done in paging and segmentation, where memory is allocated to processes non-contiguously. As a result, if you remove this condition, external fragmentation may be decreased.

ADVANTAGES

There are various advantages of fragmentation. Some of them are as follows:

- **Fast Data Writes**

Data write in a system that supports data fragmentation may be faster than reorganizing data storage to enable contiguous data writes.

- **Fewer Failures**

If there is insufficient sequential space in a system that does not support fragmentation, the write will fail.

- **Storage Optimization**

A fragmented system might potentially make better use of a storage device by utilizing every available storage block.

DISADVANTAGES

There are various disadvantages of fragmentation. Some of them are as follows:

- **Need for regular defragmentation**

A more fragmented storage device's performance will degrade with time, necessitating the requirement for time-consuming defragmentation operations.

- **Slower Read Times**

The time it takes to read a non-sequential file might increase as a storage device becomes more fragmented.

Summary -

Topic : Swapping

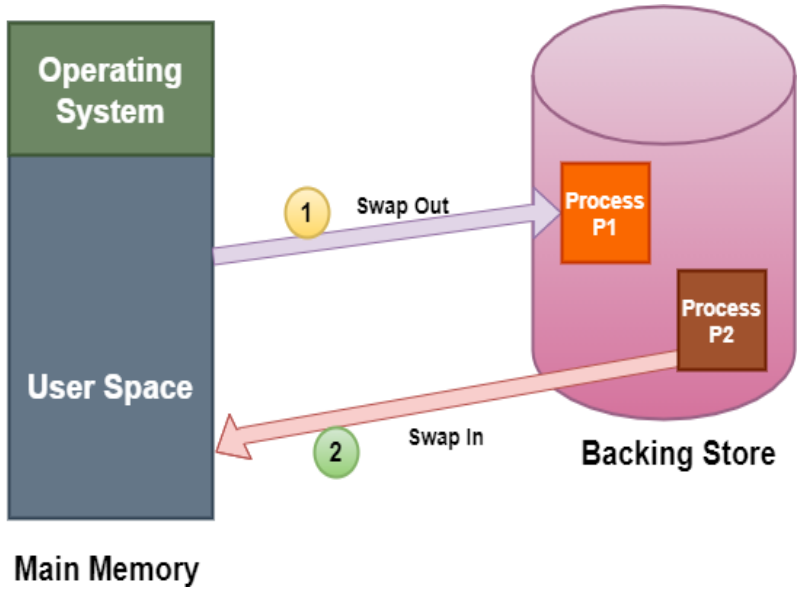
Objective :

Outcome :

Swapping in Operating System

- Swapping is a memory management scheme in which the processes are temporarily switched from the main memory to the secondary memory. It is done so that the main memory can be made available for other processes.
- The place where the swapped-out memories are kept is called swap space.
- Swapping is used when the data is not present in the RAM. It is helpful to run larger and multiple processes. Therefore it is also known as memory compaction.
- The purpose of the swapping in operating system is to access the data present in the hard disk and bring it to RAM so that the application programs can use it. The thing to remember is that swapping is used only when data is not present in RAM

Due to the swapping technique performance usually gets affected, but it also helps in running multiple and big processes in parallel. **The swapping** process is also known as a technique for **memory compaction.** Basically, low priority processes may be swapped out so that processes with a higher priority may be loaded and executed.



The above diagram shows swapping of two processes where the disk is used as a Backing store.

It is further divided into two types:

1. **Swap-in:** Swap-in is the method of removing a program from the hard disk and putting it back in the RAM.
2. **Swap-out:** Swap-out removes a program from the RAM and puts it into the hard disk.

Advantages of Swapping in OS

The advantages of the swapping method are listed as follows:

- Swapping in **OS** helps in achieving the goal of **Maximum CPU Utilization**.
- Swapping ensures proper memory availability for every process that needs to be executed.
- Swapping helps in avoiding the problem of process starvation means a process should not take much time for execution so that the next process should be executed.
- **CPU** can perform various tasks simultaneously with the help of swapping so that processes do not have to wait much longer before execution.
- Swapping ensures proper **RAM**(main memory) utilization.
- Swapping creates a dedicated disk partition in the **hard drive** for swapped processes which is called **swap space**.
- Swapping in **OS**is an economical process.
- Swapping method can be applied on **priority-based** process scheduling where a **high priority** process is swapped in and a **low priority** process is swapped out which improves the performance.

Disadvantages of Swapping in OS

There are some limited disadvantages of the swapping method which are listed as follows:

- If the system deals with **power-cut** during bulky swapping activity then the user may lose all the information which is related to the program.
- If the swapping method uses an algorithm that is not up to the mark then the number of page faults can be increased and therefore this decreases the complete performance.
- There may be inefficiency in a case when there is some common resource used by the processes that are participating in the swapping process.

Summary - Swapping in **OS** is a memory management method that temporarily swaps out idle or blocked processes from main memory to secondary memory which ensures proper memory utilization.

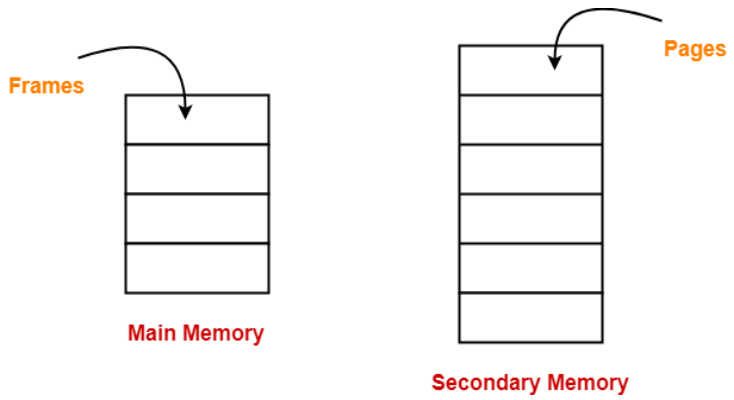
Topic : Paging

Objective :

Outcome :

PAGING

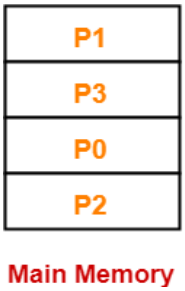
- Paging is a fixed size partitioning scheme.
- In paging, secondary memory and main memory are divided into equal fixed size partitions.
- The partitions of secondary memory are called as **pages**.
- The partitions of main memory are called as **frames**.



- Each process is divided into parts where size of each part is same as page size.
- The size of the last part may be less than the page size.
- The pages of process are stored in the frames of main memory depending upon their availability.

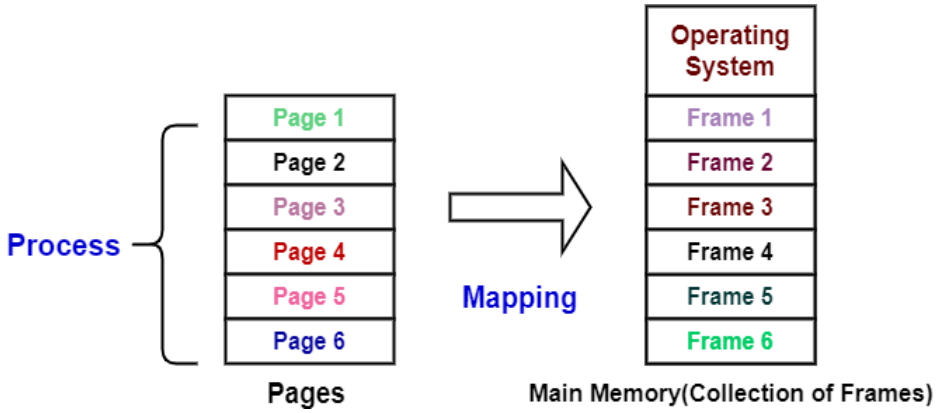
Example-

- Consider a process is divided into 4 pages P_0, P_1, P_2 and P_3.
- Depending upon the availability, these pages may be stored in the main memory frames in a non-contiguous fashion as shown-



There is a possibility that the size of the last part may be less than the page size.

- Pages of a process are brought into the main memory only when there is a requirement otherwise they reside in the secondary storage.
- One page of a process is mainly stored in one of the frames of the memory. Also, the pages can be stored at different locations of the memory but always the main priority is to find contiguous frames.



Translating Logical Address into Physical Address-

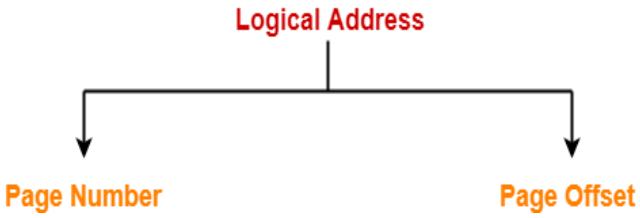
- CPU always generates a logical address.
- A physical address is needed to access the main memory.

Following steps are followed to translate logical address into physical address-

Step-01:

CPU generates a logical address consisting of two parts-

1. Page Number
2. Page Offset



- Page Number specifies the specific page of the process from which CPU wants to read the data.
- Page Offset specifies the specific word on the page that CPU wants to read.

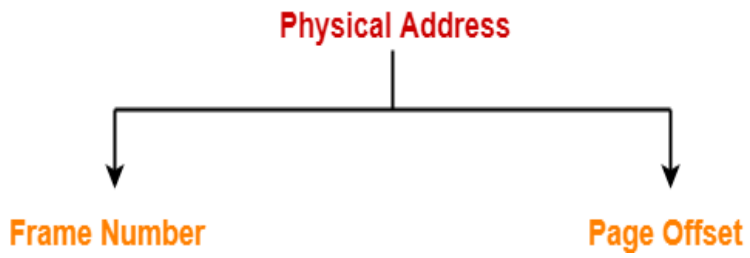
Step-02:

For the page number generated by the CPU,

- **Page Table** provides the corresponding frame number (base address of the frame) where that page is stored in the main memory.

Step-03:

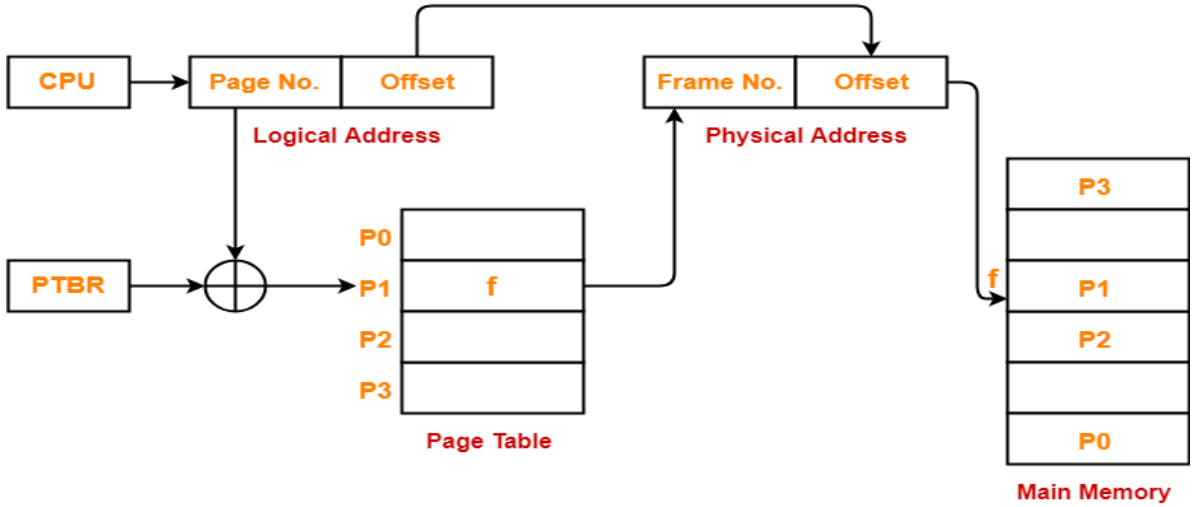
- The frame number combined with the page offset forms the required physical address.



- Frame number specifies the specific frame where the required page is stored.
- Page Offset specifies the specific word that has to be read from that page.

<u>Diagram-</u>

The following diagram illustrates the above steps of translating logical address into physical address-



Translating Logical Address into Physical Address

Advantages-

The advantages of paging are-

- It allows to store parts of a single process in a non-contiguous fashion.
- It solves the problem of external fragmentation.

Disadvantages-

- It suffers from internal fragmentation.
- There is an overhead of maintaining a page table for each process.
- The time taken to fetch the instruction increases since now two memory accesses are required.

Summary -

Topic : Segmentation

Objective :

Outcome :

Segmentation-

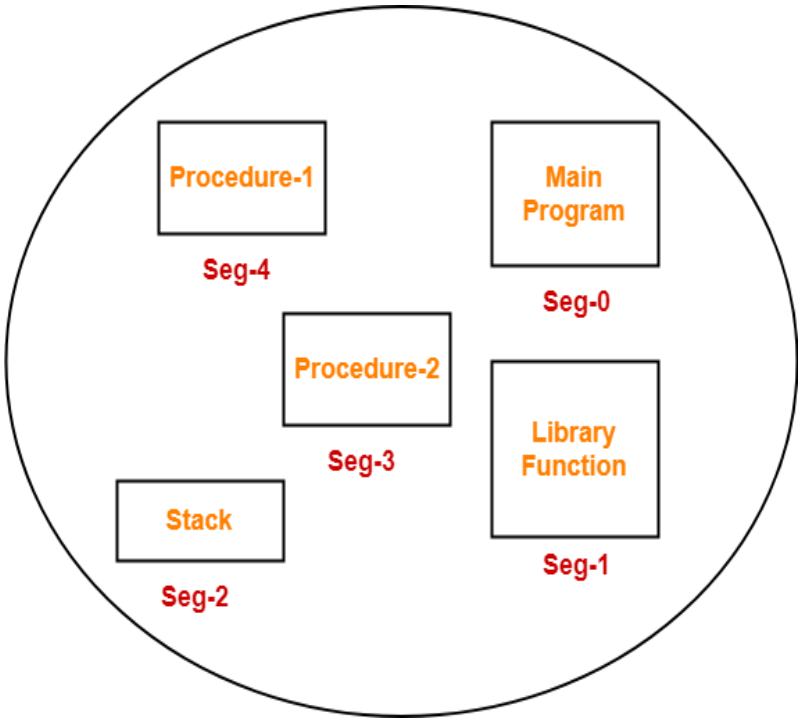
- Like Paging, Segmentation is another non-contiguous memory allocation technique.
- In segmentation, process is not divided blindly into fixed size pages.
- Segmentation is a memory management technique in which the memory is divided into the variable size parts. Each part is known as a segment which can be allocated to a process.
- Rather, the process is divided into modules for better visualization.

Characteristics-

- Segmentation is a variable size partitioning scheme.
- In segmentation, secondary memory and main memory are divided into partitions of unequal size.
- The size of partitions depend on the length of modules.
- The partitions of secondary memory are called as **segments**.

Example-

Consider a program is divided into 5 segments as-



Segment Table-

- Segment table is a table that stores the information about each segment of the process.
- It has two columns.
- First column stores the size or length of the segment.
- Second column stores the base address or starting address of the segment in the main memory.
- Segment table is stored as a separate segment in the main memory.
- Segment table base register (STBR) stores the base address of the segment table.

For the above illustration, consider the segment table is-

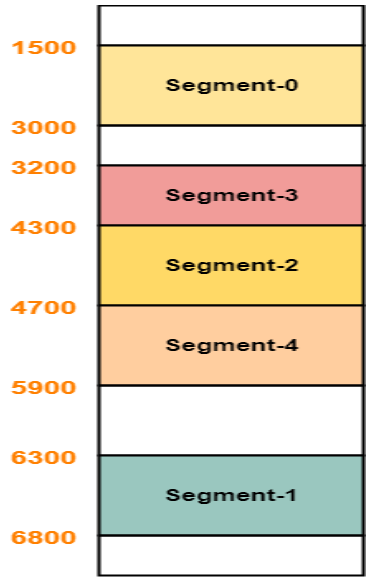
	Limit	Base
Seg-0	1500	1500
Seg-1	500	6300
Seg-2	400	4300
Seg-3	1100	3200
Seg-4	1200	4700

Segment Table

Here,

- Limit indicates the length or size of the segment.
- Base indicates the base address or starting address of the segment in the main memory.

In accordance to the above segment table, the segments are stored in the main memory as-



Main Memory

Translating Logical Address into Physical Address-

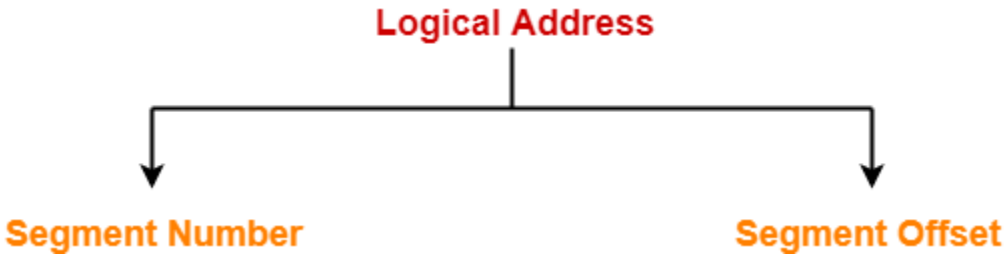
- CPU always generates a logical address.
- A physical address is needed to access the main memory.

Following steps are followed to translate logical address into physical address-

Step-01:

CPU generates a logical address consisting of two parts-

1. Segment Number
2. Segment Offset



- Segment Number specifies the specific segment of the process from which CPU wants to read the data.
- Segment Offset specifies the specific word in the segment that CPU wants to read.

Step-02:

- For the generated segment number, corresponding entry is located in the segment table.
- Then, segment offset is compared with the limit (size) of the segment.

Now, two cases are possible-

Case-01: Segment Offset >= Limit

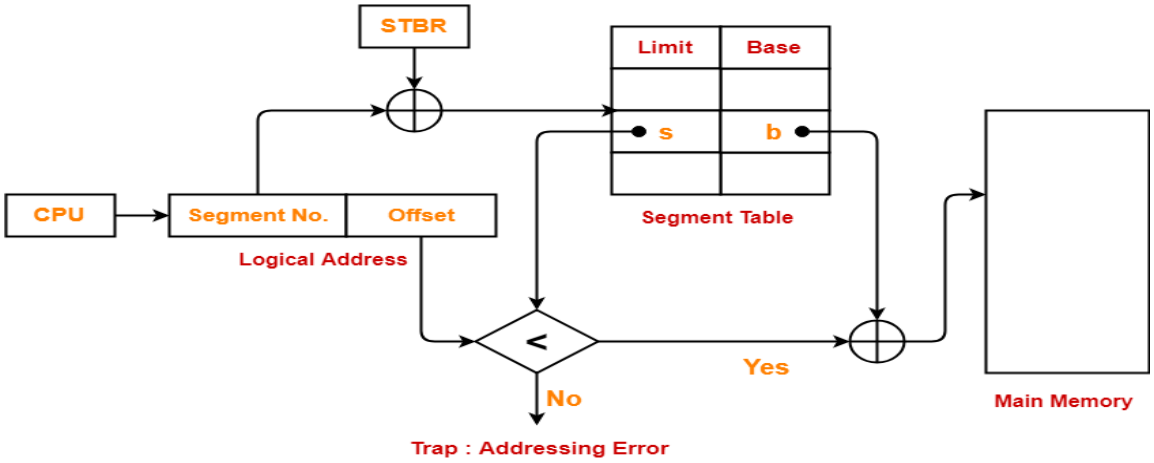
- If segment offset is found to be greater than or equal to the limit, a trap is generated.

Case-02: Segment Offset < Limit

- If segment offset is found to be smaller than the limit, then request is treated as a valid request.
- The segment offset must always lie in the range [0, limit-1],
- Then, segment offset is added with the base address of the segment.
- The result obtained after addition is the address of the memory location storing the required word.

Diagram-

The following diagram illustrates the above steps of translating logical address into physical address-



Translating Logical Address into Physical Address

Advantages-

- It allows to divide the program into modules which provides better visualization.
- Segment table consumes less space as compared to **Page Table** in paging.
- It solves the problem of internal fragmentation.

Disadvantages-

- There is an overhead of maintaining a segment table for each process.
- The time taken to fetch the instruction increases since now two memory accesses are required.
- Segments of unequal size are not suited for swapping.
- It suffers from external fragmentation as the free space gets broken down into smaller pieces with the processes being loaded and removed from the main memory.

Summary - Swapping in **OS** is a memory management method that temporarily swaps out idle or blocked processes from main memory to secondary memory which ensures proper memory utilization.

Topic : Page Fault

Objective :

Outcome :

PAGE FAULT

- When a page referenced by the CPU is not found in the main memory, it is called as a **page fault**.
- When a page fault occurs, the required page has to be fetched from the secondary memory into the main memory.

Translating Logical Address into Physical Address-

In a paging scheme using TLB with possibility of page fault,

The logical address generated by the CPU is translated into the physical address using the following steps-

Step-01:

CPU generates a logical address consisting of two parts-

1. Page Number
2. Page Offset

Step-02:

- TLB is checked to see if it contains an entry for the referenced page number.
- The referenced page number is compared with the TLB entries all at once.

Now, two cases are possible-

Case-01: If there is a TLB hit-

- If TLB contains an entry for the referenced page number, a TLB hit occurs.
- In this case, TLB entry is used to get the frame number for the referenced page number.

Case-02: If there is a TLB miss-

- If TLB does not contain an entry for the referenced page number, a TLB miss occurs.
- In this case, page table is used to get the frame number for the referenced page number.
- The valid / invalid bit of the page table entry indicates whether the referenced page is present in the main memory or not.

Now, two cases are possible-

Case-01: If Valid / Invalid Bit is Set to 1-

- If valid / invalid bit is set to 1, it indicates that the page is present in the main memory.
- Then, page table is used to get the frame number for the referenced page number.
- Then, TLB is updated with the page number and its frame number for future references.

Case-02: If Valid / Invalid Bit is Set to 0-

- If valid / invalid bit is set to 0, it indicates that the page is not present in the main memory.
- A page fault occurs.
- The occurrence of page fault calls the page fault interrupt which executes the page fault handling routine.

Page Fault Handling Routine-

The following sequence of events take place-

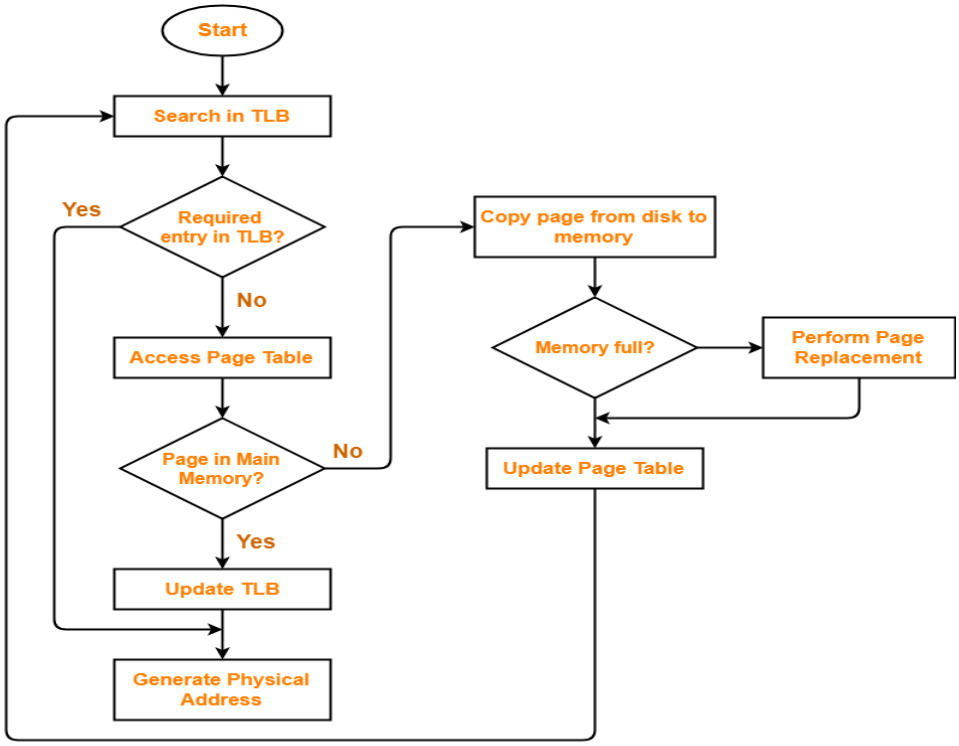
- The currently running process is stopped and context switching occurs.
- The referenced page is copied from the secondary memory to the main memory.
- If the main memory is already full, a page is replaced to create a room for the referenced page.
- After copying the referenced page successfully in the main memory, the page table is updated.
- When the execution of process is resumed, step-02 repeats.

Step-03:

- After the frame number is obtained, it is combined with the page offset to generate the physical address.
- Then, physical address is used to read the required word from the main memory.

Flowchart-

The following flowchart illustrates the above steps of translating logical address into physical address-



Flowchart

Page Fault Service Time-

- The time taken by the page fault handling routine to service the page fault is called as page fault service time.
- Page fault service time is much greater than main memory access time.
- It increases the effective access time.

Summary -

PAGE REPLACEMENT ALGORITHM

Page Replacement Algorithms-

Page replacement algorithms help to decide which page must be swapped out from the main memory to create a room for the incoming page.

Various page replacement algorithms are-

1. FIFO Page Replacement Algorithm
2. LRU Page Replacement Algorithm
3. Optimal Page Replacement Algorithm

FIFO Page Replacement Algorithm-

- As the name suggests, this algorithm works on the principle of "**First in First out**".
- It replaces the oldest page that has been present in the main memory for the longest time.
- It is implemented by keeping track of all the pages in a queue.

Advantages

- Simple to understand and implement
- Does not cause more overhead

Disadvantages

- Poor performance
- Doesn't use the frequency of the last used time and just simply replaces the oldest page.
- Suffers from Belady's anomaly.

Problem-01:

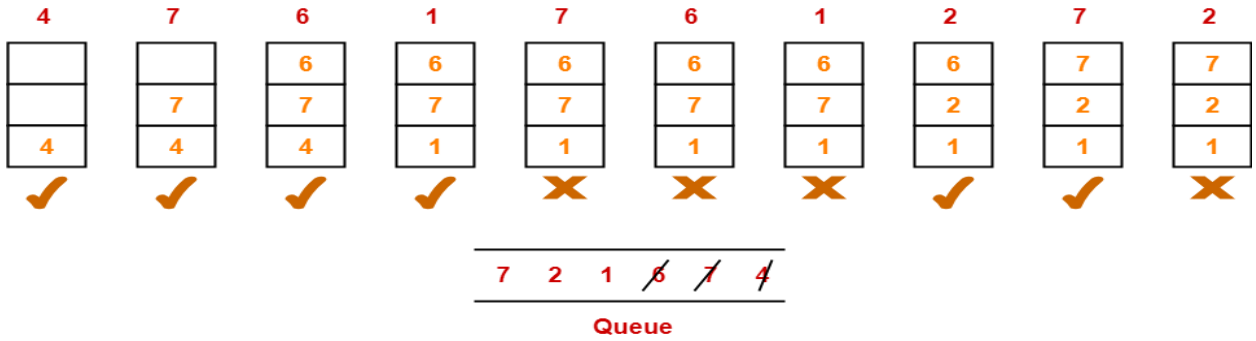
A system uses 3 page frames for storing process pages in main memory. It uses the First in First out (FIFO) page replacement policy. Assume that all the page frames are initially empty. What is the total number of page faults that will occur while processing the page reference string given below-

4 , 7, 6, 1, 7, 6, 1, 2, 7, 2

Also calculate the hit ratio and miss ratio.

Solution-

Total number of references = 10



Calculating Hit ratio-

Total number of page hits

= Total number of references – Total number of page misses or page faults

= 10 – 6

= 4

Thus, Hit ratio

= Total number of page hits / Total number of references

= 4 / 10

= 0.4 or 40%

Calculating Miss ratio-

Total number of page misses or page faults = 6

Thus, Miss ratio

= Total number of page misses / Total number of references

= 6 / 10

= 0.6 or 60%

Alternatively,

Miss ratio

= 1 – Hit ratio

= 1 – 0.4

= 0.6 or 60%

LRU Page Replacement Algorithm-

- As the name suggests, this algorithm works on the principle of "**Least Recently Used**".
- It replaces the page that has not been referred by the CPU for the longest time.

Advantages

- It is open for full analysis
- Doesn't suffer from Belady's anomaly
- Often more efficient than other algorithms

Disadvantages

- It requires additional data structures to be implemented

- More complex
- High hardware assistance is required

EXAMPLE

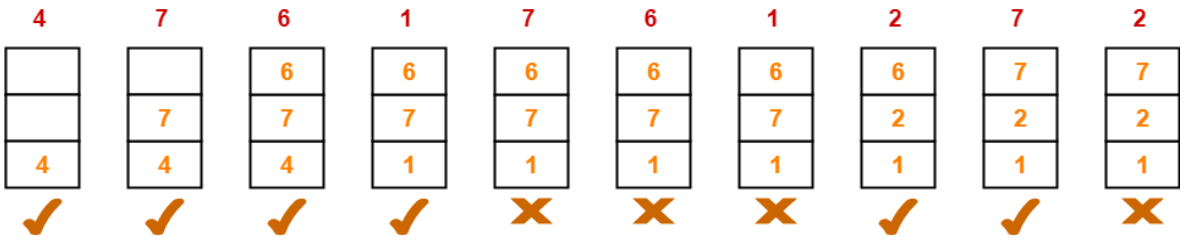
A system uses 3 page frames for storing process pages in main memory. It uses the Least Recently Used (LRU) page replacement policy. Assume that all the page frames are initially empty. What is the total number of page faults that will occur while processing the page reference string given below-

4 , 7, 6, 1, 7, 6, 1, 2, 7, 2

Also calculate the hit ratio and miss ratio.

Solution-

Total number of references = 10



From here,

Total number of page faults occurred = 6

In the similar manner as above-

- Hit ratio = 0.4 or 40%
- Miss ratio = 0.6 or 60%

Optimal Page Replacement Algorithm-

- This algorithm replaces the page that will not be referred by the CPU in future for the longest time.
- It is practically impossible to implement this algorithm.
- This is because the pages that will not be used in future for the longest time can not be predicted.
- However, it is the best known algorithm and gives the least number of page faults.
- Hence, it is used as a performance measure criterion for other algorithms.

Advantages

- Excellent efficiency
- Less complexity

- Easy to use and understand
- Simple data structures can be used to implement
- Used as the benchmark for other algorithms

Disadvantages

- More time consuming
- Difficult for error handling
- Need future awareness of the programs, which is not possible every time

Example-

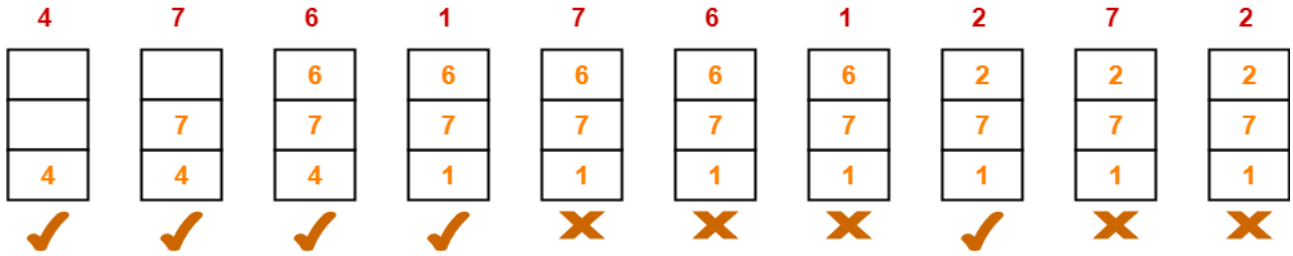
A system uses 3 page frames for storing process pages in main memory. It uses the Optimal page replacement policy. Assume that all the page frames are initially empty. What is the total number of page faults that will occur while processing the page reference string given below-

4 , 7, 6, 1, 7, 6, 1, 2, 7, 2

Also calculate the hit ratio and miss ratio.

Solution-

Total number of references = 10



From here,

Total number of page faults occurred = 5

In the similar manner as above-

- Hit ratio = 0.5 or 50%
- Miss ratio = 0.5 or 50%

Belady's Anomaly-

Belady's Anomaly is the phenomenon of increasing the number of page faults on increasing the number of frames in main memory.

NOTE-

In the above discussion,

- "Algorithms suffer from Belady's Anomaly" does not mean that always the number of page faults will increase on increasing the number of frames in main memory.
- This unusual behavior is observed only sometimes.

Reason Behind Belady's Anomaly-

- An algorithm suffers from Belady's Anomaly if and only if it does not follow stack property.
- Algorithms that follow stack property are called as **stack based algorithms**.
- Stack based algorithms do not suffer from Belady's Anomaly.
- This is because these algorithms assign priority to a page for replacement that is independent of the number of frames in the main memory.

Examples-

Following page replacement algorithms are stack based algorithms-

1. LRU Page Replacement Algorithm
2. Optimal Page Replacement Algorithm

Hence, they do not suffer from Belady's Anomaly.

Stack Property-

Consider-

- Initially, we had 'm' number of frames in the main memory.
- Now, the number of frames in the main memory is increased to 'm+1'.

According to stack property-

- At each stage, the set of pages that were present in the main memory when number of frames is 'm' will be compulsorily present in the corresponding stages in main memory when the number of frames is increased to 'm+1'.

For FIFO Page Replacement Algorithm-

Consider the reference string is-

0, 1, 2, 3, 0, 1, 4, 0, 1, 2, 3, 4

Now, observe the following two cases-

Case-01: When frame size = 3

0	1	2	3	0	1	4	0	1	2	3	4
		2	2	2	1	1	1	1	1	3	3
	1	1	1	0	0	0	0	0	2	2	2
0	0	0	3	3	3	4	4	4	4	4	4

Number of page faults = 9

Case-02: When frame size = 4

0	1	2	3	0	1	4	0	1	2	3	4
			3	3	3	3	3	3	2	2	2
		2	2	2	2	2	2	1	1	1	1
	1	1	1	1	1	1	0	0	0	0	4
0	0	0	0	0	0	4	4	4	4	3	3

Number of page faults = 10

From here, we can observe-

- At stage-07 and stage-08 in case-02, main memory does not contain the set of pages that are present in the corresponding stages in case-01.
- Thus, FIFO page replacement algorithm does not follow the stack property.
- Hence, it suffers from Belady's Anomaly.
- As a proof, number of page faults increase when the number of frames is increased from 3 to 4.